CORRIGÉ · EXERCICE · INFORMATIQUE SUP

Recherche dichotomique

Énoncé

Écrire une fonction `appartient(L, x)` qui teste si `x` est dans la liste triée `L` par dichotomie. Prouver sa terminaison et donner sa complexité.

Décoder l'énoncé

Ce que l'énoncé te donne.

- Une liste `L` triée dans l'ordre croissant, de longueur n.

Ce qu'on te demande. Un code correct, une preuve de terminaison, une complexité.

Corrigé

L'idée

On cherche `x` dans la zone `L[g..d]` (indices de `g` à `d` inclus), qui est au départ toute la liste. On regarde l'élément du milieu `L[m]` :

- s'il vaut `x`, on a trouvé ;
- s'il est plus petit que `x`, tous les éléments à sa gauche le sont aussi (liste triée) : `x` ne peut être qu'à droite de `m` ;
- s'il est plus grand, `x` ne peut être qu'à gauche.

Exemple : `L = [2, 5, 8, 12, 17, 23]`, `x = 17`. $g = 0$, $d = 5$, $m = 2$: `L[2] = 8 < 17`, donc $g = 3$. Puis $m = 4$: `L[4] = 17`, trouvé en 2 tours.

Le code

```
def appartient(L, x):
    g, d = 0, len(L) - 1
    while g <= d:               # la zone L[g..d] n'est pas vide
        m = (g + d) // 2        # indice du milieu (division entière)
        if L[m] == x:
            return True
        elif L[m] < x:
            g = m + 1           # x ne peut être qu'à droite de m
        else:
            d = m - 1           # x ne peut être qu'à gauche de m
    return False                # zone vide : x n'est pas dans L
```

On écrit `m + 1` et `m - 1` (et pas `m`) : `L[m]` a déjà été testé, on l'exclut. C'est aussi ce qui garantit que la boucle s'arrête.

Invariant : la correction

Invariant : « si `x` est dans `L`, alors il est dans `L[g..d]` ».

- **Au départ** : $g = 0$, $d = n - 1$, c'est toute la liste. Vrai.
- **Conservation** : si `L[m] < x`, comme `L` est triée, `L[0]` $\leqslant \ldots \leqslant$ `L[m] < x` : `x` n'est pas dans `L[g..m]`, il est donc dans `L[m+1..d]`. Le cas `L[m] > x` est symétrique.
- **À la sortie** : si on renvoie `True`, c'est que `L[m] == x`. Si la boucle s'arrête, $g > d$: la zone est vide, donc par l'invariant `x` n'est pas dans `L`, et `False` est correct.

Variant : la terminaison

On prend $v = d - g + 1$, le nombre d'éléments de la zone. Tant que la boucle tourne, $g \leqslant d$, donc $v \geqslant 1$: c'est un entier positif.

À chaque tour, $g \leqslant m \leqslant d$. Si `g = m + 1`, la nouvelle taille est $d - (m+1) + 1 = d - m < d - g + 1$. Si `d = m - 1`, elle vaut $(m-1) - g + 1 = m - g < d - g + 1$. Donc v **diminue strictement**.

Une suite d'entiers positifs strictement décroissante est finie : la boucle s'arrête.

La complexité

Avec $v = d - g + 1$ et $m = \lfloor \frac{g+d}{2} \rfloor$, la nouvelle taille vaut $d - m$ ou $m - g$, et ces deux nombres sont $\leqslant \frac{d-g}{2} = \frac{v-1}{2} < \frac{v}{2}$. La taille est donc **au moins divisée par** 2 à chaque tour.

Après k tours, la taille est $< \frac{n}{2^k}$. La boucle tourne tant que la taille est $\geqslant 1$, donc il faut $\frac{n}{2^k} > 1$, soit $2^k < n$, c'est-à-dire $k < \log_2 n$. Il y a donc au plus $\lfloor \log_2 n \rfloor + 1$ tours, chacun en temps constant :

complexité $O(\log n)$.

Ordre de grandeur : pour $n = 10^6$, au plus 20 tours, contre jusqu'à 10^6 comparaisons pour une recherche élément par élément.

Le piège

Écrire `g = m` au lieu de `g = m + 1` : la boucle peut ne jamais terminer.

La méthode

Pour toute boucle `while`, donne un variant pour la terminaison et un invariant pour la correction.

Rappels de cours utilisés

- **Invariant de boucle.** Une propriété vraie avant la boucle et conservée par chaque tour. À la sortie, elle prouve la correction de l'algorithme.
- **Variant de boucle.** Un entier positif qui décroît strictement à chaque tour. Son existence prouve que la boucle termine.
- **Complexité logarithmique.** Si la taille du problème est au moins divisée par 2 à chaque étape, le nombre d'étapes est au plus $\lfloor \log_2 n \rfloor + 1$, soit $O(\log n)$.